

C++ Interview Question Bank

OOP topics, then STL topics — 5 Predict-the-Output + 5 MCQ per topic

Topic Index

OOP

1. Classes & Objects / Access Specifiers
2. Constructors & Destructors
3. Inheritance & Access Control
4. Polymorphism — Compile-time (Overloading)
5. Polymorphism — Runtime (virtual, vtable, slicing)
6. Static Members & Friend Functions
7. Templates
8. Exception Handling
9. Modern C++ (constexpr, smart pointers, move, lambdas, optional)

STL

10. Vector
11. Priority Queue / Heap
12. Map / Set / Pair
13. Algorithms (sort, bounds, accumulate, count_if)
14. Iterator Pitfalls

Topic 1: Classes & Objects / Access Specifiers

Predict the Output

1. What is the output of the following code?

```
class Test {
public:
    int score;
    Test(int num) { score = num; }
    void printScore() { cout << "Score: " << score << endl; }
};
int main() {
    Test t(42);
    t.printScore();
}
```

Answer: Score: 42

2. What is the output of the following code?

```
struct S { int x; void show(){cout<<x;} };
class C { int x=5; public: void show(){cout<<x;} };
int main(){
    S s; s.x=10; s.show(); cout<<" ";
    C c; c.show();
}
```

Answer: 10 5

3. What is the output of the following code?

```
class Box { int side; };
int main() { Box b; b.side = 5; }
```

Answer: Compilation error — 'side' is private within this context (struct/class members default to private in a class).

4. What is the output of the following code?

```
class Inner { public: Inner(){cout<<"Inner ";} };
class Outer { Inner in; public: Outer(){cout<<"Outer ";} };
int main(){ Outer o; }
```

Answer: Inner Outer

5. What is the output of the following code?

```
class Point { int x; public: Point(int x){this->x=x;} void show(){cout<<this->x;} };
int main(){ Point p(7); p.show(); }
```

Answer: 7

Multiple Choice

1. What is the default access specifier for members of a class (not struct)?

- a) public
- b) protected
- c) private

d) none — must specify

Answer: c) private

2. struct vs class differ in:

- a) struct can't have member functions
- b) the default access specifier only (public vs private)
- c) struct can't be inherited from
- d) class must have a constructor

Answer: b) the default access specifier only (public vs private)

3. Which of the following can access a private member of a class from outside the class?

- a) Any global function
- b) A derived class's member function
- c) A friend function
- d) Any function in the same file

Answer: c) A friend function

4. A C++ namespace is primarily used to:

- a) improve runtime performance
- b) avoid name collisions across code
- c) enable multithreading
- d) enforce access control

Answer: b) avoid name collisions across code

5. What does the `this` pointer refer to inside a non-static member function?

- a) The class itself
- b) The calling object
- c) A null pointer by default
- d) The base class subobject

Answer: b) The calling object

Topic 2: Constructors & Destructors

Predict the Output

1. What is the output of the following code?

```
class Base{public: Base(){cout<<"Base ";}};
class Derived: public Base{public: Derived(){cout<<"Derived ";}};
int main(){ Derived d; }
```

Answer: Base Derived

2. What is the output of the following code?

```
class Base{public: ~Base(){cout<<"~Base ";}};
class Derived: public Base{public: ~Derived(){cout<<"~Derived ";}};
int main(){ Derived d; }
```

Answer: ~Derived ~Base

3. What is the output of the following code?

```
class Buf { public: int *data; Buf(int v){ data=new int(v);} };
int main(){
    Buf a(10);
    Buf b = a;          // compiler-generated (shallow) copy
    *b.data = 99;
    cout << *a.data;
}
```

Answer: 99 — the default copy constructor copies the pointer, not the pointed-to int, so a and b share the same heap block.

4. What is the output of the following code?

```
class C{ public: C(){cout<<"C ";} C(const C&){cout<<"Copy ";} };
void f(C c){}
int main(){ C c1; f(c1); }
```

Answer: C Copy — c1 is constructed, then passing it by value to f() invokes the copy constructor.

5. What is the output of the following code?

```
class A {
    int x, y;
public:
    A(int v): y(v), x(y) { cout << x << " " << y; }
};
int main(){ A a(5); }
```

Answer: x prints as garbage/indeterminate, y prints 5. Members initialize in declaration order (x then y) regardless of the order written in the initializer list, so x(y) reads y before it's set.

Multiple Choice

1. When is a class's destructor called automatically for a local (stack) object?

- a) At program exit only
- b) When the object goes out of scope
- c) Only if explicitly invoked

d) Never — delete must be called

Answer: b) When the object goes out of scope

2. If a class has a raw pointer member and no user-defined copy constructor, copying an object with the compiler-generated copy constructor produces:

- a) A deep copy
- b) A shallow copy (the pointer is copied, not the pointee)
- c) A compile error
- d) A move instead of a copy

Answer: b) A shallow copy (the pointer is copied, not the pointee)

3. In `class C: public A, public B`, the order base-class constructors run in is:

- a) Reverse declaration order
- b) The order the base classes appear in the derived class's base-list
- c) Alphabetical order
- d) Unspecified/undefined

Answer: b) The order the base classes appear in the derived class's base-list

4. A class with only a private constructor:

- a) can never be instantiated, even internally
- b) can only be instantiated via its own member/static/friend functions — common in the singleton pattern
- c) is a compile error at the class definition itself
- d) behaves identically to a public constructor

Answer: b) can only be instantiated via its own member/static/friend functions — common in the singleton pattern

5. If a base class has a virtual destructor, a derived class that doesn't declare its own destructor:

- a) causes a compile error
- b) still gets correct cleanup via the base pointer — the derived destructor is implicitly virtual too
- c) leads to undefined behavior on every delete
- d) only ever runs the base destructor

Answer: b) still gets correct cleanup via the base pointer — the derived destructor is implicitly virtual too

Topic 3: Inheritance & Access Control

Predict the Output

1. What is the output of the following code?

```
class Base{ protected: int x=5; };
class Derived: public Base{ public: void show(){cout<<x;} };
int main(){ Derived d; d.show(); }
```

Answer: 5

2. What is the output of the following code?

```
class Base{ public: int x=1; };
class Derived: private Base{ public: void show(){ cout<<x; } };
int main(){ Derived d; d.show(); }
```

Answer: 1 — private inheritance only restricts access from outside Derived; Derived's own member functions still see x as accessible.

3. What is the output of the following code?

```
class Base{ public: int x=1; };
class Derived: private Base{};
int main() { Derived d; d.x = 2; }
```

Answer: Compilation error — private inheritance makes Base's public members private within Derived, so x is inaccessible from main().

4. What is the output of the following code?

```
class A{ public: int x=1; };
class B: public A{};
class C: public A{};
class D: public B, public C{};
int main(){ D d; cout << d.x; }
```

Answer: Compilation error — request for member 'x' is ambiguous. D contains two separate A subobjects (one via B, one via C).

5. What is the output of the following code?

```
class A{ public: int x=1; };
class B: virtual public A{};
class C: virtual public A{};
class D: public B, public C{};
int main(){ D d; cout << d.x; }
```

Answer: 1 — virtual inheritance makes B and C share a single A subobject, resolving the diamond ambiguity.

Multiple Choice

1. Which inheritance access specifier keeps base-class public members public in the derived class?

- a) private
- b) protected
- c) public
- d) there is no default

Answer: c) public

2. A protected member of a base class is accessible from:

- a) anywhere in the program
- b) only within the base class
- c) the base class and its derived classes (not from outside objects)
- d) only friend functions

Answer: c) the base class and its derived classes (not from outside objects)

3. The 'diamond problem' in multiple inheritance refers to:

- a) circular #includes causing a build failure
- b) a derived class inheriting the same base twice through two paths, causing ambiguous member access
- c) a class inheriting from itself
- d) having too many virtual functions

Answer: b) a derived class inheriting the same base twice through two paths, causing ambiguous member access

4. Can a derived class's member function directly access a private member declared in its base class?

- a) Yes, always
- b) Yes, but only with public inheritance
- c) No — private members are never directly accessible from a derived class
- d) Only if the derived class is implicitly a friend

Answer: c) No — private members are never directly accessible from a derived class

5. What problem does 'virtual inheritance' solve?

- a) runtime polymorphism for functions
- b) duplicate base-class subobjects in diamond inheritance
- c) access-specifier conflicts
- d) template instantiation ambiguity

Answer: b) duplicate base-class subobjects in diamond inheritance

Topic 4: Polymorphism — Compile-time (Overloading)

Predict the Output

1. What is the output of the following code?

```
void f(int x){cout<<"int ";}  
void f(double x){cout<<"double ";}  
int main(){ f(5); f(5.5); f('a'); }
```

Answer: int double int — 'a' promotes to int, matching f(int).

2. What is the output of the following code?

```
void f(int &x){ cout << "lvalue ref "; }  
void f(int &&x){ cout << "rvalue ref "; }  
int main(){ int a=5; f(a); f(10); }
```

Answer: lvalue ref rvalue ref

3. What is the output of the following code?

```
class Animal{ public: void speak(){cout<<"Animal ";} };  
class Dog: public Animal{ public: void speak(int x){cout<<"Dog ";} };  
int main(){ Dog d; d.speak(); }
```

Answer: Compilation error — Dog::speak(int) hides Animal::speak() by name, so the no-argument speak() isn't found on a Dog object.

4. What is the output of the following code?

```
class C{ public:  
    void show(){ cout<<"non-const "; }  
    void show() const { cout<<"const "; }  
};  
int main(){ C c; c.show(); const C c2; c2.show(); }
```

Answer: non-const const — const-ness of the object is part of overload resolution.

5. What is the output of the following code?

```
class Vec{ public: int x; Vec(int x):x(x){}  
    Vec operator+(const Vec&o){ return Vec(x+o.x); } };  
int main(){ Vec a(3), b(4); Vec c=a+b; cout<<c.x; }
```

Answer: 7

Multiple Choice

1. Function overloading is resolved:

- a) at runtime, based on the object's dynamic type
- b) at compile time, based on argument types/count
- c) at link time
- d) only for member functions, never free functions

Answer: b) at compile time, based on argument types/count

2. Two functions differing ONLY in return type (identical parameters):

- a) is legal overloading

- b) is a compile error — return type alone can't distinguish overloads
- c) is resolved by the caller's assignment type
- d) produces a linker warning only

Answer: b) is a compile error — return type alone can't distinguish overloads

3. If a derived class declares a function with the same name but a different signature than a base class function, the base version in the derived class's scope is:

- a) still visible via unqualified calls
- b) hidden (name hiding), unless brought in with a using-declaration
- c) immediately a compile error
- d) auto-overloaded by the compiler

Answer: b) hidden (name hiding), unless brought in with a using-declaration

4. Which is true about operator overloading in C++?

- a) every operator can be overloaded
- b) most operators can be overloaded, but a few (e.g. `::`, `.`, `sizeof`, `?:`) cannot
- c) only arithmetic operators can be overloaded
- d) overloaded operators must be member functions

Answer: b) most operators can be overloaded, but a few (e.g. `::`, `.`, `sizeof`, `?:`) cannot

5. Calling `f(5)` when both `f(int&)` and `f(const int&)` are declared:

- a) is ambiguous — compile error
- b) prefers `f(int&)`
- c) prefers `f(const int&)`, since `int&` can't bind to an rvalue
- d) always calls `f(int&)` via a temporary

Answer: c) prefers `f(const int&)`, since `int&` can't bind to an rvalue

Topic 5: Polymorphism — Runtime (virtual, vtable, slicing)

Predict the Output

1. What is the output of the following code?

```
class Animal{ public: virtual void speak(){cout<<"Animal ";} };
class Dog: public Animal{ public: void speak() override {cout<<"Dog ";} };
int main(){ Animal* a = new Dog(); a->speak(); }
```

Answer: Dog

2. What is the output of the following code?

```
class Animal{ public: virtual void speak(){cout<<"Animal ";} };
class Dog: public Animal{ public: void speak() override {cout<<"Dog ";} };
int main(){ Dog d; Animal a = d; a.speak(); }
```

Answer: Animal — object slicing. Assigning a Dog to an Animal by value copies only the Animal part, so 'a' is statically bound to Animal::speak().

3. What is the output of the following code?

```
class Shape{ public: virtual void draw() = 0; };
int main(){ Shape s; }
```

Answer: Compilation error — cannot declare a variable of abstract type 'Shape'.

4. What is the output of the following code?

```
class Base{ public: ~Base(){cout<<"~Base ";} };
class Derived: public Base{ public: ~Derived(){cout<<"~Derived ";} };
int main(){ Base* b = new Derived(); delete b; }
```

Answer: ~Base — Base's destructor isn't virtual, so deleting through a Base* skips Derived's destructor (undefined behavior in general; in practice here only ~Base runs).

5. What is the output of the following code?

```
class Base{ public: Base(){ init(); } virtual void init(){cout<<"Base::init ";} };
class Derived: public Base{ public: void init() override {cout<<"Derived::init ";} };
int main(){ Derived d; }
```

Answer: Base::init — a virtual call made during a base class's own constructor uses the base's version; the derived part doesn't exist yet.

Multiple Choice

1. A vtable (virtual table) is:

- a) created per-object at runtime, for every object
- b) a per-class table of function pointers used to resolve virtual calls, shared across instances
- c) used only for pure virtual functions
- d) a compile-time-only construct with no runtime presence

Answer: b) a per-class table of function pointers used to resolve virtual calls, shared across instances

2. "Object slicing" refers to:

- a) splitting an object across memory pages

- b) assigning a derived-class object to a base-class object by value, losing the derived part
- c) a compiler optimization for virtual calls
- d) manually deleting part of an object's memory

Answer: b) assigning a derived-class object to a base-class object by value, losing the derived part

3. A pure virtual function is declared as:

- a) virtual void f();
- b) virtual void f() = 0;
- c) virtual void f() {}
- d) abstract void f();

Answer: b) virtual void f() = 0;

4. Calling a virtual function from inside a base class constructor:

- a) dispatches to the most-derived override, as usual
- b) dispatches only to the base class's own version, since the derived part isn't constructed yet
- c) is a compile error
- d) is always undefined behavior

Answer: b) dispatches only to the base class's own version, since the derived part isn't constructed yet

5. The main risk of giving a base class a non-virtual destructor when it's used polymorphically is:

- a) slower destructor calls
- b) deleting a derived object through a base pointer skips the derived destructor — resource leaks / UB
- c) a compile error at every delete
- d) vtable corruption

Answer: b) deleting a derived object through a base pointer skips the derived destructor — resource leaks / UB

Topic 6: Static Members & Friend Functions

Predict the Output

1. What is the output of the following code?

```
class Test{ public: static int count;
    Test& fun(){ count++; cout<<count<<" "; return *this; } };
int Test::count = 0;
int main(){ Test t1; t1.fun().fun().fun().fun(); Test t2; t2.fun(); }
```

Answer: 1 2 3 4 5 — count is shared across all instances, so t2 continues the same counter.

2. What is the output of the following code?

```
class Box{ int side; public: Box(int s):side(s){} friend void show(Box b); };
void show(Box b){ cout << b.side; }
int main(){ Box b(9); show(b); }
```

Answer: 9

3. What is the output of the following code?

```
void counter(){ static int n = 0; n++; cout << n << " "; }
int main(){ counter(); counter(); counter(); }
```

Answer: 1 2 3 — a function-local static persists (and keeps its value) across calls, initialized only once.

4. What is the output of the following code?

```
class Base{ int x=1; friend void peek(Base&); };
class Derived: public Base{};
void peek(Base& b){ cout << b.x; }
int main(){ Derived d; peek(d); }
```

Answer: 1 — peek is a friend of Base; a Derived& binds to Base& via the inherited subobject, so the private Base::x is still reachable.

5. What is the output of the following code?

```
class Container{ int capacity;
public:
    Container(int c):capacity(c){}
    friend void show(); };
void Container::show(){ }
int main(){ }
```

Answer: Compilation error — 'show' was declared as a non-member friend function, so it cannot be defined with 'Container::' scope resolution (that syntax is only valid for actual members).

Multiple Choice

1. A static data member of a class:

- a) has one copy per object
- b) has a single copy shared by all objects of the class
- c) cannot be modified after compilation
- d) is only accessible via a friend function

Answer: b) has a single copy shared by all objects of the class

2. A friend function of a class:

- a) is a member function with public access
- b) is a non-member (or another class's member) function granted access to the class's private/protected members
- c) must be declared virtual
- d) is automatically inherited by derived classes

Answer: b) is a non-member (or another class's member) function granted access to the class's private/protected members

3. Where must a static data member typically be defined (pre-C++17, without `inline`)?

- a) only inside the class definition
- b) outside the class, in exactly one translation unit (e.g. `int Class::x = 0;`)
- c) it needs no definition, only a declaration
- d) in `main()`

Answer: b) outside the class, in exactly one translation unit (e.g. `int Class::x = 0;`)

4. Is friendship inherited by derived classes, or transitive between friends?

- a) yes to both
- b) no to both — friendship is neither inherited nor transitive
- c) only inherited, not transitive
- d) only transitive, not inherited

Answer: b) no to both — friendship is neither inherited nor transitive

5. A static member function:

- a) can access non-static members directly via `this`
- b) has no `this` pointer and can only directly access other static members
- c) must always be called through an object
- d) runs once automatically at program start

Answer: b) has no `this` pointer and can only directly access other static members

Topic 7: Templates

Predict the Output

1. What is the output of the following code?

```
template<class T> T maxVal(T a, T b){ return a>b? a:b; }
int main(){ cout << maxVal(3,7) << " " << maxVal(2.5, 1.5); }
```

Answer: 7 2.5

2. What is the output of the following code?

```
template <class T>
class A{ public: T func(T a, T b){ return (a*1.0)/b; } };
int main(){ A<int> a1; cout << a1.func(3,2) << " " << a1.func(3.0,2.0); }
```

Answer: 1 1 — A<int> fixes T=int, so the return value truncates to int both times; the second call's 3.0/2.0 arguments are also converted to int (3, 2) before the call.

3. What is the output of the following code?

```
template<class T> void show(T x){ cout << "generic "; }
template<> void show<char>(char x){ cout << "char-special "; }
int main(){ show(5); show('a'); }
```

Answer: generic char-special

4. What is the output of the following code?

```
template<int N> int square(){ return N*N; }
int main(){ cout << square<5>(); }
```

Answer: 25

5. What is the output of the following code?

```
template<class T> T add(T a, T b){ return a+b; }
struct Point{ int x,y; };
int main(){ Point p1{1,2}, p2{3,4}; Point p3 = add(p1,p2); }
```

Answer: Compilation error — Point has no operator+, so add<Point> fails to instantiate.

Multiple Choice

1. Templates achieve polymorphism at:

- a) runtime, via vtables
- b) compile time, via code generation for each instantiated type
- c) link time only
- d) they don't provide polymorphism

Answer: b) compile time, via code generation for each instantiated type

2. What actually triggers a template's compilation/instantiation?

- a) simply writing the template definition
- b) the compiler seeing the template declared in a header
- c) using the template with a concrete type (implicit or explicit instantiation)
- d) linking the final binary

Answer: c) using the template with a concrete type (implicit or explicit instantiation)

3. A class template instantiated with different type arguments means:

- a) one binary version exists for all types
- b) the compiler generates separate code per type used (A<int> and A<double> are distinct classes)
- c) it always uses void* internally
- d) templates can't be used for classes, only functions

Answer: b) the compiler generates separate code per type used (A<int> and A<double> are distinct classes)

4. Template specialization is used to:

- a) speed up compilation only
- b) provide a custom implementation for a specific type, overriding the generic template
- c) make a template private
- d) disable a template for every type

Answer: b) provide a custom implementation for a specific type, overriding the generic template

5. A non-type template parameter (e.g. `template<int N>`) must be:

- a) any runtime-computed value
- b) a compile-time constant expression
- c) only an integer literal typed directly, never constexpr
- d) not permitted in C++

Answer: b) a compile-time constant expression

Topic 8: Exception Handling

Predict the Output

1. What is the output of the following code?

```
void check(int s){ if (s>100) throw out_of_range("speed exceeded"); }
int main(){
    try{ check(100); check(150); check(120); }
    catch(const out_of_range& e){ cout << "Caught: " << e.what(); }
    cout << " Done";
}
```

Answer: Caught: speed exceeded Done — check(150) throws and is caught; check(120) is never reached.

2. What is the output of the following code?

```
int main(){
    int x=-1;
    try{ if(x<0) throw x; }
    catch(...){ cout << "Exception occurred, exiting"; }
}
```

Answer: Exception occurred, exiting

3. What is the output of the following code?

```
int main(){
    try{ throw 5; }
    catch(const char* e){ cout << "caught string"; }
}
```

Answer: Program terminates abnormally (std::terminate) — a thrown int has no matching catch(const char*) handler.

4. What is the output of the following code?

```
try{ throw runtime_error("oops"); }
catch(const exception& e){ cout << "base: " << e.what(); }
catch(const runtime_error& e){ cout << "derived: " << e.what(); }
```

Answer: base: oops — the first matching handler wins. runtime_error derives from exception, so the base catch takes it; g++ actually warns that the second catch is unreachable.

5. What is the output of the following code?

```
void f(){ try{ throw 1; } catch(int){ cout << "inner "; throw; } }
int main(){ try{ f(); } catch(int){ cout << "outer"; } }
```

Answer: inner outer — the bare 'throw;' re-throws the same exception up to main's handler.

Multiple Choice

1. A 'catch(...)' block:

- a) catches only standard exceptions
- b) catches any thrown exception, but you can't inspect its type/value
- c) must be the first catch block
- d) is a syntax error

Answer: b) catches any thrown exception, but you can't inspect its type/value

2. If a thrown exception matches no catch block and propagates out of main:

- a) it's silently ignored
- b) the program calls `std::terminate()`
- c) it converts to a default int exception
- d) it's a compile-time error

Answer: b) the program calls `std::terminate()`

3. Multiple catch blocks for one try are evaluated:

- a) in the order written — first match wins
- b) by picking the most specific type automatically
- c) in reverse order
- d) all matching blocks run

Answer: a) in the order written — first match wins

4. A bare `throw;` (no operand) inside a catch block:

- a) throws a new empty exception
- b) re-throws the currently caught exception, preserving its original type
- c) is a syntax error outside a function
- d) clears the exception and continues

Answer: b) re-throws the currently caught exception, preserving its original type

5. An exception thrown from a destructor while another exception is already propagating (stack unwinding):

- a) is handled normally like any throw
- b) can call `std::terminate()` if it escapes the destructor — two exceptions can't propagate at once
- c) is automatically ignored
- d) is converted to a warning

Answer: b) can call `std::terminate()` if it escapes the destructor — two exceptions can't propagate at once

Topic 9: Modern C++ (constexpr, smart pointers, move, lambdas, optional)

Predict the Output

1. What is the output of the following code?

```
constexpr int square(int x){ return x*x; }
int main(){ constexpr int r = square(5); cout << r; }
```

Answer: 25

2. What is the output of the following code?

```
unique_ptr<int> p1 = make_unique<int>(10);
unique_ptr<int> p2 = move(p1);
cout << (p1==nullptr) << " " << *p2;
```

Answer: 1 10 — move leaves p1 null and transfers ownership to p2.

3. What is the output of the following code?

```
shared_ptr<int> p1 = make_shared<int>(5);
{ shared_ptr<int> p2 = p1; cout << p1.use_count() << " "; }
cout << p1.use_count();
```

Answer: 2 1 — p2 bumps the ref count to 2 while in scope; it drops back to 1 once p2 is destroyed.

4. What is the output of the following code?

```
int x=10;
auto byVal = [x]() { return x; };
auto byRef = [&x]() { return x; };
x = 20;
cout << byVal() << " " << byRef();
```

Answer: 10 20 — [x] captures a snapshot at lambda-creation time; [&x] reads the live variable.

5. What is the output of the following code?

```
optional<int> find(bool ok){ if(ok) return 42; return nullopt; }
int main(){
    auto r = find(false);
    cout << (r.has_value() ? "has value" : "empty") << " " << r.value_or(-1);
}
```

Answer: empty -1

Multiple Choice

1. `constexpr` means:

- a) the value is decided only at runtime, for safety
- b) the expression can be evaluated at compile time (when its inputs allow it)
- c) it's identical to `const`, just newer syntax
- d) it only applies to member functions

Answer: b) the expression can be evaluated at compile time (when its inputs allow it)

2. `std::move(x)` internally:

- a) physically relocates x in memory
- b) casts x to an rvalue reference, enabling move semantics if available
- c) deep-copies x
- d) destroys x immediately

Answer: b) casts x to an rvalue reference, enabling move semantics if available

3. `std::unique_ptr` differs from `std::shared_ptr` in that:

- a) `unique_ptr` allows multiple owners, `shared_ptr` allows only one
- b) `unique_ptr` enforces sole ownership (non-copyable, only movable); `shared_ptr` allows shared ownership via ref-counting
- c) they are functionally identical
- d) `shared_ptr` can't be used with arrays

Answer: b) `unique_ptr` enforces sole ownership (non-copyable, only movable); `shared_ptr` allows shared ownership via ref-counting

4. A lambda that captures by reference (`&x`) and is used after `x` goes out of scope:

- a) is always safe since C++11
- b) risks a dangling reference — undefined behavior if the lambda outlives x
- c) auto-copies x at that point
- d) is a compile-time error

Answer: b) risks a dangling reference — undefined behavior if the lambda outlives x

5. `std::variant` (C++17) is best described as:

- a) a dynamically-typed container, like a Python variable
- b) a type-safe union holding one value from a fixed set of alternative types at a time
- c) a synonym for `void*`
- d) usable only with numeric types

Answer: b) a type-safe union holding one value from a fixed set of alternative types at a time

Topic 10: Vector

Predict the Output

1. What is the output of the following code?

```
vector<int> v;  
for(int i=1;i<=3;i++) v.push_back(i);  
cout << v.size() << " " << v.capacity();
```

Answer: 3 4 — size is exactly 3; libstdc++'s growth (doubling from 1) leaves capacity at 4 after the 3rd push_back. Capacity is implementation-defined; only size() is guaranteed.

2. What is the output of the following code?

```
vector<int> v(5); cout << v.size() << " " << v[0];
```

Answer: 5 0 — vector<int>(5) value-initializes all 5 elements to 0.

3. What is the output of the following code?

```
vector<int> v = {1,2,3};  
v.reserve(3);  
auto it = v.begin();  
v.push_back(4);  
cout << *it;
```

Answer: Undefined behavior — push_back beyond the reserved capacity (3) forces a reallocation, invalidating 'it'.

4. What is the output of the following code?

```
vector<vector<int>> v(2, vector<int>(3, 7));  
cout << v[1][2];
```

Answer: 7

5. What is the output of the following code?

```
vector<int> v = {1,2,3,2,4,2};  
v.erase(remove(v.begin(), v.end(), 2), v.end());  
for(int x: v) cout << x << " ";
```

Answer: 1 3 4 — the erase-remove idiom.

Multiple Choice

1. `vector::push\_back` has time complexity of:

- a) O(n) always
- b) O(1) amortized
- c) O(log n)
- d) O(n²)

Answer: b) O(1) amortized

2. When a vector reallocates during growth, existing iterators/pointers/references into it:

- a) remain valid
- b) are invalidated
- c) only pointers are invalidated, iterators stay valid
- d) only end() is invalidated

Answer: b) are invalidated

3. `vector<int> v(5, 3);` creates:

- a) an empty vector with capacity 5
- b) a vector of 5 elements, all initialized to 3
- c) a vector containing the two elements 5 and 3
- d) a compile error

Answer: b) a vector of 5 elements, all initialized to 3

4. `vector::size()` vs `vector::capacity()`:

- a) always equal
- b) `size()` is the current element count; `capacity()` is allocated storage, always $\geq$ `size()`
- c) `capacity()` is always less than `size()`
- d) they're interchangeable

Answer: b) `size()` is the current element count; `capacity()` is allocated storage, always $\geq$ `size()`

5. Which STL container does NOT support random access via `[]`?

- a) vector
- b) deque
- c) list
- d) array

Answer: c) list

Topic 11: Priority Queue / Heap

Predict the Output

1. What is the output of the following code?

```
priority_queue<int> pq;
pq.push(3); pq.push(7); pq.push(1);
cout << pq.top();
```

Answer: 7 — default priority_queue is a max-heap.

2. What is the output of the following code?

```
priority_queue<int, vector<int>, greater<int>> pq;
pq.push(3); pq.push(7); pq.push(1);
cout << pq.top();
```

Answer: 1 — greater<int> turns it into a min-heap.

3. What is the output of the following code?

```
vector<int> v = {5,1,9,3};
priority_queue<int> pq(v.begin(), v.end());
while(!pq.empty()){ cout << pq.top() << " "; pq.pop(); }
```

Answer: 9 5 3 1

4. What is the output of the following code?

```
priority_queue<int> pq;
pq.pop();
```

Answer: Undefined behavior — popping an empty priority_queue is not checked or exception-safe, unlike some other containers' bounds-checked accessors.

5. What is the output of the following code?

```
priority_queue<pair<int,int>> pq;
pq.push({1,100}); pq.push({2,50}); pq.push({1,200});
cout << pq.top().first << " " << pq.top().second;
```

Answer: 2 50 — pair comparison is lexicographic; {2,50} beats {1,x} on the first element alone.

Multiple Choice

1. `priority_queue::top()` has time complexity:

- a) $O(\log n)$
- b) $O(1)$
- c) $O(n)$
- d) $O(n^2)$

Answer: b) $O(1)$

2. `priority_queue::push()` has time complexity:

- a) $O(1)$ amortized
- b) $O(\log n)$
- c) $O(n)$
- d) $O(n \log n)$

Answer: b) $O(\log n)$

3. Constructing a priority_queue directly from a range, `priority_queue<int> pq(v.begin(), v.end())`, runs in:

- a) $O(n \log n)$ always
- b) $O(n)$ — uses `make_heap` internally
- c) $O(n^2)$
- d) $O(\log n)$

Answer: b) $O(n)$ — uses `make_heap` internally

4. By default, `std::priority_queue<int>` behaves as:

- a) a min-heap (smallest on top)
- b) a max-heap (largest on top)
- c) a FIFO queue
- d) a sorted array

Answer: b) a max-heap (largest on top)

5. Which of these does `std::priority_queue` NOT support directly?

- a) `push()`
- b) `pop()`
- c) `top()`
- d) random-access iteration or insertion in the middle

Answer: d) random-access iteration or insertion in the middle

Topic 12: Map / Set / Pair

Predict the Output

1. What is the output of the following code?

```
map<string,int> m;  
cout << m["missing"] << " " << m.size();
```

Answer: 0 1 — operator[] on a missing key default-constructs int() (=0) and inserts it.

2. What is the output of the following code?

```
map<int,string> m;  
m[3]="c"; m[1]="a"; m[2]="b";  
for(auto& p: m) cout << p.first << p.second << " ";
```

Answer: 1a 2b 3c — std::map keeps keys sorted.

3. What is the output of the following code?

```
set<int> s = {5,3,5,1,3};  
cout << s.size() << " ";  
for(int x: s) cout << x << " ";
```

Answer: 3 1 3 5 — duplicates are silently dropped; remaining elements print sorted.

4. What is the output of the following code?

```
pair<int,int> a = {1,5};  
pair<int,int> b = {1,3};  
cout << (a > b);
```

Answer: 1 — first elements tie, so the comparison falls through to the second (5>3).

5. What is the output of the following code?

```
unordered_map<int,int> m;  
m[1]=10; m[2]=20; m[3]=30;  
cout << m.size();
```

Answer: 3 — size() is deterministic even though iteration order isn't.

Multiple Choice

1. `std::map` internally stores its elements:

- a) in insertion order
- b) sorted by key (typically a balanced BST like a red-black tree)
- c) in hash-bucket order
- d) in reverse insertion order

Answer: b) sorted by key (typically a balanced BST like a red-black tree)

2. Accessing a non-existent key via `map::operator[]`:

- a) throws an exception
- b) returns a reference and inserts a default-constructed value for that key
- c) returns nullptr
- d) is a compile error

Answer: b) returns a reference and inserts a default-constructed value for that key

3. `std::set` differs from `std::multiset` in that:

- a) set allows duplicates, multiset doesn't
- b) set doesn't allow duplicate elements; multiset does
- c) set is unsorted, multiset is sorted
- d) there is no difference

Answer: b) set doesn't allow duplicate elements; multiset does

4. `std::unordered_map` gives average time complexity for lookup of:

- a) $O(\log n)$
- b) $O(1)$ average, $O(n)$ worst case
- c) $O(n)$ always
- d) $O(n \log n)$

Answer: b) $O(1)$ average, $O(n)$ worst case

5. `std::pair<int,int>`'s relational operators (`<`, `>`, ...) compare:

- a) only the first element
- b) only the second element
- c) lexicographically — first element first, second as tiebreaker
- d) by memory address

Answer: c) lexicographically — first element first, second as tiebreaker

Topic 13: Algorithms (sort, bounds, accumulate, count_if)

Predict the Output

1. What is the output of the following code?

```
vector<int> v = {2,1,3,5,4};
sort(v.begin(), v.end(), greater<int>());
for(int x: v) cout << x << " ";
```

Answer: 5 4 3 2 1

2. What is the output of the following code?

```
vector<int> v = {10,20,30,40,50};
cout << *upper_bound(v.begin(), v.end(), 30) << " " << *lower_bound(v.begin(),
v.end(), 30);
```

Answer: 40 30 — upper_bound finds the first element > 30; lower_bound finds the first element ≥ 30.

3. What is the output of the following code?

```
vector<int> v = {1,2,3,4,5};
cout << accumulate(v.begin(), v.end(), 0);
```

Answer: 15

4. What is the output of the following code?

```
vector<int> v = {1,2,3,4,5,6};
int c = count_if(v.begin(), v.end(), [](int x){ return x%2==0; });
cout << c;
```

Answer: 3

5. What is the output of the following code?

```
class Solution{ public:
    bool comp(int a, int b){ return a<b; }
    void run(vector<int>& v){ sort(v.begin(), v.end(), comp); }
};
int main(){ Solution s; vector<int> v={3,1,2}; s.run(v); }
```

Answer: Compilation error — comp is a non-static member function; sort() needs a plain callable, not a bound member (fix: make it static, or pass a lambda).

Multiple Choice

1. `std::sort` has average/worst-case time complexity of:

- a) $O(n)$
- b) $O(n \log n)$
- c) $O(n^2)$
- d) $O(\log n)$

Answer: b) $O(n \log n)$

2. `upper\_bound(first, last, val)` on a sorted range returns an iterator to:

- a) the first element equal to val
- b) the first element strictly greater than val

- c) the last element less than val
- d) the element closest to val

Answer: b) the first element strictly greater than val

3. `lower_bound(first, last, val)` returns an iterator to:

- a) the first element greater than val
- b) the first element $\geq$ val
- c) the last element $\leq$ val
- d) always the same as `upper_bound`

Answer: b) the first element $\geq$ val

4. `std::accumulate` (from `<numeric>`) requires:

- a) a sorted range
- b) an initial value and a range; folds elements with `+` (or a supplied binary op)
- c) only integer types
- d) a comparator function

Answer: b) an initial value and a range; folds elements with `+` (or a supplied binary op)

5. `count_if` differs from `count` in that:

- a) `count_if` is faster
- b) `count_if` takes a predicate to test each element; `count` checks equality to a fixed value
- c) `count_if` only works on sorted ranges
- d) they're identical

Answer: b) `count_if` takes a predicate to test each element; `count` checks equality to a fixed value

Topic 14: Iterator Pitfalls

Predict the Output

1. What is the output of the following code?

```
vector<int> v = {1,3,3,5};
for(auto it=v.begin(); it!=v.end(); ++it){
    if(*it==3) it = v.erase(it);
}
for(int x: v) cout << x << " ";
```

Answer: 1 3 5 — erase() returns the next valid iterator, but the loop's own ++it then skips over it, so one of the two 3's survives unremoved.

2. What is the output of the following code?

```
vector<int> v = {1,2,3};
v.reserve(3);
int& ref = v[0];
v.push_back(4);
cout << ref;
```

Answer: Undefined behavior — push_back beyond the reserved capacity reallocates the buffer, so ref dangles.

3. What is the output of the following code?

```
vector<int> v = {1,2,3};
for(int& x : v) x *= 2;
for(int x : v) cout << x << " ";
```

Answer: 2 4 6

4. What is the output of the following code?

```
map<int,int> m = {{1,10},{2,20},{3,30}};
auto it = m.find(2);
auto it2 = m.begin();
m.erase(it);
cout << it2->first;
```

Answer: 1 — erasing from a map only invalidates the erased iterator; it2 remains valid.

5. What is the output of the following code?

```
int& getRef(){
    int local = 42;
    return local;
}
int main(){
    int& r = getRef();
    cout << r;
}
```

Answer: Undefined behavior — local is destroyed when getRef() returns, so r dangles (it may happen to print 42, or garbage).

Multiple Choice

1. The correct, safe pattern for erasing elements from a vector while iterating is:

- a) `for(it=v.begin(); ...; ++it) if(cond) v.erase(it);`
- b) reassign the iterator from `erase()`'s return value instead of blindly incrementing (e.g. ``it = cond ? v.erase(it) : ++it;``)
- c) `erase` always combines safely with a plain `++it`
- d) use a range-based for loop and `erase` inside it

Answer: b) reassign the iterator from `erase()`'s return value instead of blindly incrementing (e.g. ``it = cond ? v.erase(it) : ++it;``)

2. Which containers guarantee erasing an element does NOT invalidate other iterators (besides the erased one)?

- a) vector and deque
- b) `std::list`, `std::map`, `std::set`
- c) all STL containers
- d) none

Answer: b) `std::list`, `std::map`, `std::set`

3. Returning a reference to a local (stack) variable from a function:

- a) is safe as long as the function is short
- b) produces a dangling reference — undefined behavior when the caller uses it
- c) is a compile-time error
- d) extends the local's lifetime automatically

Answer: b) produces a dangling reference — undefined behavior when the caller uses it

4. Reserving capacity with ``vector::reserve(n)`` protects existing pointers/references to elements from invalidation as long as:

- a) you never call `clear()`
- b) the vector's size never exceeds the reserved capacity `n`
- c) you call `push_back()` only once
- d) it never protects them

Answer: b) the vector's size never exceeds the reserved capacity `n`

5. In ``for (int& x : v)``, modifying ``x`` inside the loop body:

- a) has no effect on the original container
- b) modifies the actual element in `v`, since `x` is a reference
- c) only works for vectors, not other containers
- d) requires `const`

Answer: b) modifies the actual element in `v`, since `x` is a reference